Gebze Technical University
Department of Computer Engineering

CSE344 – System Programming
Homework #2

Signal Handling & Process Management with `fork()`

Name: [Salih Cengiz]
Student ID: [220104004007]
Course: CSE344 – System Programming
Date: March 2026

Contents

1 Introduction **2**
1.1 Key Features . . . 2

2 Design Decisions **2**
2.1 Overall Architecture . . . 2
2.2 Directory Partitioning . . . 2
2.3 Pattern Matching Algorithm . . . 3
2.4 Signal Handling Strategy . . . 3
2.4.1 Async-Signal-Safety Measures . . . 3
2.4.2 Signal Flow . . . 4
2.5 Worker-Parent Communication . . . 4
2.6 Wait Loop Design . . . 4

3 Challenges and Solutions **5**
3.1 Challenge 1: Parent Hanging in Wait Loop . . . 5
3.2 Challenge 2: Greedy Pattern Matching Without Backtracking . . . 5
3.3 Challenge 3: Duplicate Notice Messages After fork() . . . 5
3.4 Challenge 4: Stack Overflow Risk . . . 5
3.5 Challenge 5: Collecting Exit Status for Zero-Match Workers . . . 5

4 Mandatory Test Scenario **6**
4.1 Step 1: Test directory tree under `/tmp/crawltest` . . . 6
4.2 Step 2: Basic search with `rep+ort` . . . 6
4.3 Step 3: Size filter `-s 15` . . . 7
4.4 Step 4: Pattern with no matches . . . 7
4.5 Step 5: Large directory, `SIGINT`, and zombie check . . . 7
4.6 Step 6: `stderr` / log evidence for signal-related behavior . . . 8

5 Conclusion **9**

1 Introduction

This report describes the design and implementation of `procSearch`, a multi-process file search tool written in C for Linux. The program accepts a root directory and a filename pattern, forks a configurable number of worker processes, and assigns each worker a partition of the directory tree to search recursively. Workers report results to the parent through signals, and the parent produces a final tree-formatted output with a summary.

The program is built with a Makefile, compiles with `-Wall` with zero warnings, and runs on Debian 11 (64-bit). All inter-process communication relies on POSIX signals (`SIGUSR1`, `SIGINT`, `SIGTERM`, `SIGCHLD`) and temporary files for match data transfer.

1.1 Key Features

- Custom pattern matching with `+` operator (one or more of the preceding character), implemented from scratch without any regex library
- Round-robin directory partitioning among worker processes
- Full signal handling with `sigaction()` and async-signal-safe handlers
- Tree-formatted output with per-worker match summary
- Zombie process prevention via `SIGCHLD` handler
- Graceful shutdown on `SIGINT` (Ctrl+C)

2 Design Decisions

2.1 Overall Architecture

The program follows a parent-worker model:

```
Parent Process
  |
  |-- fork() --> Worker 0  (assigned: alpha, delta, ...)
  |-- fork() --> Worker 1  (assigned: beta, ...)
  |-- fork() --> Worker 2  (assigned: gamma, ...)
  |
  |-- wait for SIGUSR1 from all workers
  |-- collect results from temp files
  |-- print tree output and summary
```

The parent is responsible for argument parsing (`getopt`), directory partitioning, forking workers, waiting for completion, and producing the final output. Workers perform the actual recursive directory traversal and pattern matching.

2.2 Directory Partitioning

The `read_subdirectories()` function reads the immediate children of the root directory using `opendir()/readdir()`. Only directories (checked via `stat()` and `S_ISDIR`) are collected. These subdirectories are distributed among workers using round-robin assignment:

```
subdirectory[i] --> worker[i % num_workers]
```

Two edge cases are handled:

- **Fewer subdirectories than workers:** The worker count is reduced to match the subdirectory count, and a notice is printed.
- **No subdirectories at all:** The parent searches the root directory directly via `parent_direct_search()` without forking any workers.

2.3 Pattern Matching Algorithm

The `match_pattern()` / `match_pattern_at()` functions implement a custom pattern matching engine supporting the `+` operator. The algorithm performs substring matching (the pattern can match anywhere within the filename) and is case-insensitive (using `tolower()`).

The `+` operator applies to the character immediately to its left and means "one or more occurrences." For example, `rep+ort` matches `report`, `repport`, `reppport`, etc.

A key design decision was implementing **backtracking** for the `+` operator. When a `c+` group is encountered:

1. All consecutive occurrences of `c` in the filename are consumed (greedy).
2. The remaining pattern is tested against the remaining filename.
3. If the test fails, the number of consumed characters is reduced by one and retried.
4. This continues until either a match is found or the minimum of 1 repetition is reached.

This backtracking is essential for patterns like `er+ro+r` matching `error`, where `r+` must not consume both `r`s so that the subsequent literal `r` in the pattern can also match.

The backtracking is implemented via recursion in `match_pattern_at()` — when a `+` group is encountered, the function calls itself with the remaining pattern and each possible consumption length.

2.4 Signal Handling Strategy

All signal handlers are installed using `sigaction()` (not `signal()`) for reliable, portable behavior. The handlers are minimal and async-signal-safe:

Signal	Handler	Behavior
`SIGUSR1`	`handle_sigusr1`	Increments `finished_worker_count` (`volatile sig_atomic_t`)
`SIGINT`	`handle_sigint`	Sets `sigint_received` flag
`SIGTERM`	`handle_sigterm`	Sets `sigterm_received` flag (worker-side)
`SIGCHLD`	`handle_sigchld`	Calls `waitpid(-1, &status, WNOHANG)` in a loop, records PID/status

2.4.1 Async-Signal-Safety Measures

- **No `printf()`, `malloc()`, or `free()` inside any handler.** All handlers only modify `volatile sig_atomic_t` variables or call `waitpid()` (which is async-signal-safe).
- **`errno` preservation:** The `SIGCHLD` handler saves and restores `errno` to avoid corrupting the main program's error state.

- **`SA_RESTART` is intentionally not used** for `SIGUSR1` and `SIGCHLD` handlers. This ensures that `sleep()` in the parent's wait loop is interrupted when a signal arrives, allowing the loop condition to be re-evaluated immediately.
- **`SA_NOCLDSTOP`** is set for `SIGCHLD` to avoid spurious signals when children are stopped (not terminated).

2.4.2 Signal Flow

```
Worker finishes  --> SIGUSR1 --> Parent (increment counter)
User Ctrl+C      --> SIGINT  --> Parent (send SIGTERM to all
   workers)
Parent SIGTERM   --> SIGTERM --> Worker (stop, report partial
   results)
Worker exits     --> SIGCHLD --> Parent (reap zombie, record
   status)
```

2.5 Worker-Parent Communication

Workers communicate match results to the parent through temporary files:

- **Match data:** `/tmp/procSearch_<PID>.tmp` — one line per match in `path|size|worker_pid` format
- **Scan count:** `/tmp/procSearch_<PID>_info.tmp` — contains the number of files scanned

This approach avoids the complexity of pipes or shared memory while being safe across process boundaries. The parent reads these files after all workers have finished and deletes them afterward.

2.6 Wait Loop Design

The parent waits for all workers using a `while` loop with `sleep(1)`:

```
while (finished_worker_count < total_worker_count &&
       reaped_count < total_worker_count &&
       !sigint_received) {
    sleep(1);
}
```

The loop checks two independent completion indicators:

- `finished_worker_count`: incremented by `SIGUSR1` handler
- `reaped_count`: incremented by `SIGCHLD` handler

This dual-condition approach provides robustness: even if a `SIGUSR1` signal is lost (e.g., due to signal coalescing), the `SIGCHLD`-based count ensures the parent eventually exits the loop.

3 Challenges and Solutions

3.1 Challenge 1: Parent Hanging in Wait Loop

Problem: The parent process would hang indefinitely in the wait loop, even after all workers had finished.

Root Cause: `SA_RESTART` was set for `SIGUSR1` and `SIGCHLD` handlers. When a signal arrived during `sleep(1)`, the kernel automatically restarted the `sleep()` call instead of returning, preventing the loop from re-evaluating its condition.

Solution: Removed `SA_RESTART` from `SIGUSR1` and `SIGCHLD` signal configurations. Now `sleep()` is interrupted by any signal, the function returns early, and the loop condition is checked immediately.

3.2 Challenge 2: Greedy Pattern Matching Without Backtracking

Problem: The `+` operator consumed all matching characters greedily with no way to try fewer matches. For example, `er+ro+r` failed to match `error` because `r+` consumed both `r` characters, leaving none for the subsequent literal `r` in the pattern.

Solution: Implemented backtracking in `match_pattern_at()`. When a `c+` group is encountered, the function first finds the maximum number of matches, then tries each possible consumption count from maximum down to 1 by recursively calling itself with the remaining pattern.

3.3 Challenge 3: Duplicate Notice Messages After fork()

Problem: The notice message was printed multiple times — once by the parent and once by each forked child.

Root Cause: `stdout` is fully buffered when redirected to a pipe. The `printf()` output remained in the buffer when `fork()` was called, and each child inherited a copy of the buffer. When each child exited, its copy was flushed, producing duplicate output.

Solution: Added `fflush(stdout)` immediately after printing the notice message and before any `fork()` call.

3.4 Challenge 4: Stack Overflow Risk

Problem: Several large arrays were allocated on the stack, risking stack overflow.

Solution: Large arrays in `main()` and helper functions were declared `static`, moving them from the stack to the data segment. Additionally, `MAX_DIRS_PER_WORKER` was set to 512 to limit per-worker memory footprint.

3.5 Challenge 5: Collecting Exit Status for Zero-Match Workers

Problem: Workers with 0 matches called `exit(0)`. The fallback `waitpid()` loop used `match_count > 0` to determine if a worker was already collected, causing zero-match workers to be redundantly processed.

Solution: Added a `collected` flag to the `WorkerInfo` struct. This flag is set when a worker's exit status is processed, regardless of the match count value.

4 Mandatory Test Scenario

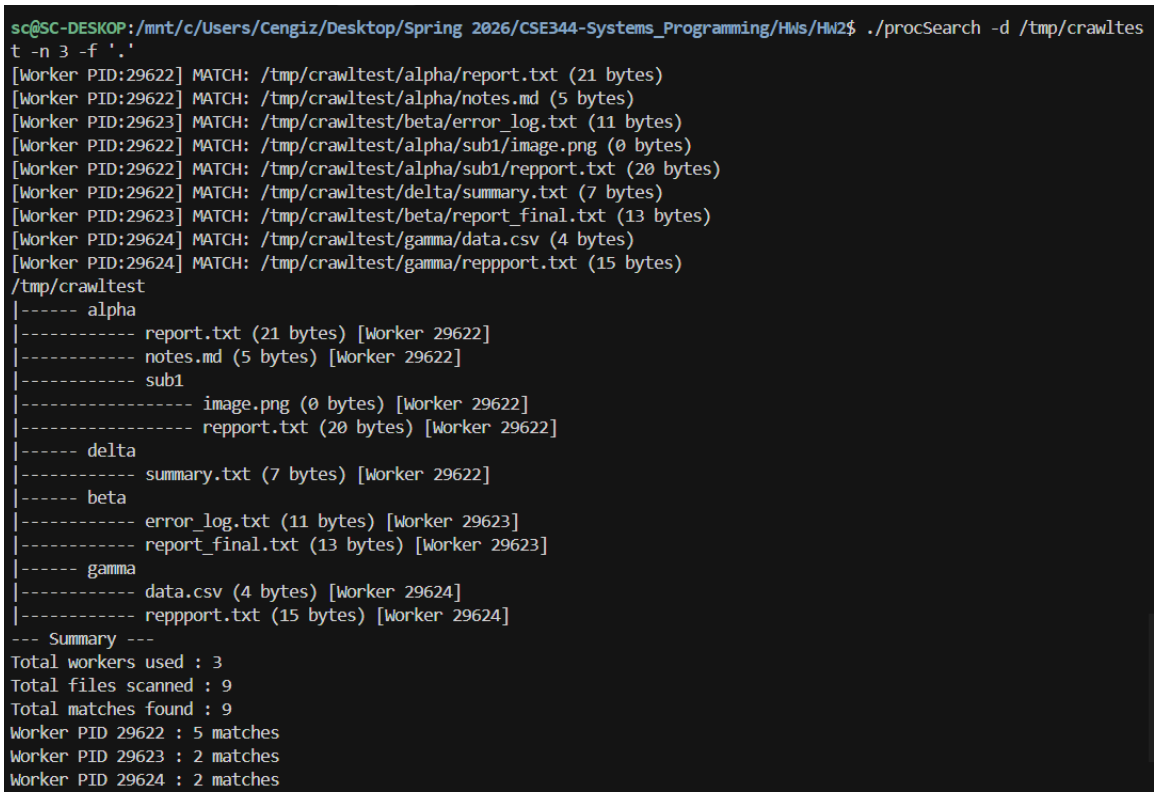
This section follows the assignment PDF *Mandatory Test Scenario*. Screenshots are loaded from the `screenshots/` folder next to `report.tex`. Compile with:

```
cd <HW2_directory>
pdflatex report.tex
pdflatex report.tex
```

For each step, the first existing file among `StepN`, `stepN`, or `step-0N` with extension `.png` or `.pdf` is included. Rename your captures to match, or adjust the macro arguments in the source.

4.1 Step 1: Test directory tree under `/tmp/crawltest`

Create the directory structure and files as specified (e.g. `mkdir`, `echo`, `touch`). Capture terminal output showing the tree or `find`/`ls` listing.



```
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$ ./procSearch -d /tmp/crawltes
t -n 3 -f '.'
[Worker PID:29622] MATCH: /tmp/crawltest/alpha/report.txt (21 bytes)
[Worker PID:29622] MATCH: /tmp/crawltest/alpha/notes.md (5 bytes)
[Worker PID:29623] MATCH: /tmp/crawltest/beta/error_log.txt (11 bytes)
[Worker PID:29622] MATCH: /tmp/crawltest/alpha/sub1/image.png (0 bytes)
[Worker PID:29622] MATCH: /tmp/crawltest/alpha/sub1/repport.txt (20 bytes)
[Worker PID:29622] MATCH: /tmp/crawltest/delta/summary.txt (7 bytes)
[Worker PID:29623] MATCH: /tmp/crawltest/beta/report_final.txt (13 bytes)
[Worker PID:29624] MATCH: /tmp/crawltest/gamma/data.csv (4 bytes)
[Worker PID:29624] MATCH: /tmp/crawltest/gamma/reppport.txt (15 bytes)
/tmp/crawltest
|------ alpha
|------------ report.txt (21 bytes) [Worker 29622]
|------------ notes.md (5 bytes) [Worker 29622]
|------------ sub1
|------------------ image.png (0 bytes) [Worker 29622]
|------------------ repport.txt (20 bytes) [Worker 29622]
|------ delta
|------------ summary.txt (7 bytes) [Worker 29622]
|------ beta
|------------ error_log.txt (11 bytes) [Worker 29623]
|------------ report_final.txt (13 bytes) [Worker 29623]
|------ gamma
|------------ data.csv (4 bytes) [Worker 29624]
|------------ reppport.txt (15 bytes) [Worker 29624]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 9
Worker PID 29622 : 5 matches
Worker PID 29623 : 2 matches
Worker PID 29624 : 2 matches
```

Figure 1: Step 1: Directory tree and files under `/tmp/crawltest` as required by the assignment.

4.2 Step 2: Basic search with `rep+ort`

Command (adjust pattern per instructor revision if needed):

```
./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort'
```

```
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$ ./procSearch -d /tmp/crawltes
t -n 3 -f 'rep+ort'
[Worker PID:29543] MATCH: /tmp/crawltest/alpha/report.txt (21 bytes)
[Worker PID:29544] MATCH: /tmp/crawltest/beta/report_final.txt (13 bytes)
[Worker PID:29543] MATCH: /tmp/crawltest/alpha/sub1/repport.txt (20 bytes)
[Worker PID:29545] MATCH: /tmp/crawltest/gamma/reppport.txt (15 bytes)
/tmp/crawltest
|------ alpha
|------------ report.txt (21 bytes) [Worker 29543]
|------------ sub1
|----------------- repport.txt (20 bytes) [Worker 29543]
|------ beta
|------------ report_final.txt (13 bytes) [Worker 29544]
|------ gamma
|------------ reppport.txt (15 bytes) [Worker 29545]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 4
Worker PID 29543 : 2 matches
Worker PID 29544 : 1 match
Worker PID 29545 : 1 match
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$
```

Figure 2: Step 2: `procSearch` run showing live `MATCH` lines, tree output, and summary.

4.3 Step 3: Size filter `-s 15`

```
./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' -s 15
```

```
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$ ./procSearch -d /tmp/crawltes
t -n 3 -f 'rep+ort' -s 15
[Worker PID:29559] MATCH: /tmp/crawltest/alpha/report.txt (21 bytes)
[Worker PID:29559] MATCH: /tmp/crawltest/alpha/sub1/repport.txt (20 bytes)
[Worker PID:29561] MATCH: /tmp/crawltest/gamma/reppport.txt (15 bytes)
/tmp/crawltest
|------ alpha
|------------ report.txt (21 bytes) [Worker 29559]
|------------ sub1
|----------------- repport.txt (20 bytes) [Worker 29559]
|------ gamma
|------------ reppport.txt (15 bytes) [Worker 29561]
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 3
Worker PID 29559 : 2 matches
Worker PID 29560 : 0 matches
Worker PID 29561 : 1 match
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$
```

Figure 3: Step 3: Only files with size $\geq$ 15 bytes appear in results.

4.4 Step 4: Pattern with no matches

```
./procSearch -d /tmp/crawltest -n 3 -f 'xyz+123'
```

4.5 Step 5: Large directory, `SIGINT`, and zombie check

```
./procSearch -d /usr/share/doc -n 4 -f 'rep+ort'
# While running: Ctrl+C, then:
ps -ef | grep procSearch | grep -v grep
```

```
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$ ./procSearch -d /tmp/crawltes
t -n 3 -f 'xyz+123'
No matching files found.
--- Summary ---
Total workers used : 3
Total files scanned : 9
Total matches found : 0
Worker PID 29590 : 0 matches
Worker PID 29591 : 0 matches
Worker PID 29592 : 0 matches
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$
```

Figure 4: Step 4: Output shows `No matching files found.` and correct summary.

```
|------------ apt-get (51680 bytes) [Worker 4780]
|------------ gdk-pixbuf-pixdata (14656 bytes) [Worker 4780]
|------------ cpan (8360 bytes) [Worker 4780]
|------------ apt (18824 bytes) [Worker 4780]
|------------ pyserial-ports (969 bytes) [Worker 4780]
|------------ systemd-delta (27000 bytes) [Worker 4780]
|------------ shasum (9979 bytes) [Worker 4780]
|------------ crontab (39568 bytes) [Worker 4780]
--- Summary ---
Total workers used : 2
Total files scanned : 0
Total matches found : 0
Worker PID 4779 : 0 matches
Worker PID 4780 : 0 matches
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$ ps -ef | grep procSearch
sc          4782     799  0 19:09 pts/0    00:00:00 grep --color=auto procSearch
sc@SC-DESKOP:/mnt/c/Users/Cengiz/Desktop/Spring 2026/CSE344-Systems_Programming/HWs/HW2$
```

Figure 5: Step 5: `SIGINT` shutdown message, clean termination, and/or `ps` showing no leftover/zombie `procSearch` processes.

4.6 Step 6: `stderr` / log evidence for signal-related behavior

Example (redirect stderr while testing):

```
./procSearch -d /tmp/crawltest -n 3 -f 'rep+ort' 2> run.stderr
# Or strace (optional):
strace -f -e trace=signal -o strace.log ./procSearch ...
```

```
run.stderr
[monitor] session start, parent PID 8369, worker count 4
[monitor] SIGCHLD: reaped child PID 8371 (exited, code 1)
[monitor] SIGCHLD: reaped child PID 8372 (exited, code 2)
[monitor] SIGCHLD: reaped child PID 8373 (exited, code 4)
[monitor] SIGCHLD: reaped child PID 8370 (exited, code 17)
Ctrl+L to chat, Ctrl+K to generate
```

Figure 6: Step 6: `stderr` or terminal capture relevant to `SIGINT` handling and/or unexpected worker termination (`SIGCHLD` path), as applicable to your run.

5 Conclusion

The `procSearch` tool successfully implements all required features: custom pattern matching with backtracking, round-robin directory partitioning, multi-process search with `fork()`, comprehensive signal handling, and tree-formatted output. The program compiles with `-Wall` and zero warnings, correctly prevents zombie processes, and handles edge cases such as insufficient subdirectories and graceful shutdown via Ctrl+C. All signal handlers are async-signal-safe, using only `volatile sig_atomic_t` variables and async-signal-safe system calls.